

DEUXIÈME PHASE DU PROJET

Service Abonnements

Fonctionnalités critiques, plan de tests, tableau de synthèse et scénario fonctionnel complet — microservice indépendant, formules et souscriptions, branché sur le vrai backend.

Système de billetterie intelligente

ReactJS · Node.js / Express · MySQL

Juillet 2026

Service Abonnements

Projet : Système de billetterie intelligente avec QR Code et abonnements. Périmètre : Service Abonnements — gestion des formules, souscription d'un client, consommation des voyages et cycle de vie d'un abonnement. Deuxième phase du projet, démarrée avec l'accord du professeur une fois le Service Utilisateurs terminé (voir [TP1-service-utilisateurs.md](#)).

Même répartition qu'au premier TP : une personne sur le front (`frontend/src/**`), une sur le back, cette fois un microservice indépendant (`service-abonnements/` , Node/Express/MySQL, port 5060) qui ne touche jamais à la base MongoDB du Service Utilisateurs — seul le jeton JWT est partagé entre les deux services. Le contrat d'API a été figé avant d'écrire une ligne de code : [PLAN-SERVICE-ABONNEMENTS.md](#).

1. Fonctionnalités critiques

Intégrité d'un abonnement — si ça casse, un client paie sans avoir de titre valide, ou voyage sans avoir payé : - Souscription à une formule : la date d'expiration et le solde de voyages sont calculés côté backend au moment de la souscription, jamais recalculés côté front. - Un client ne peut avoir qu'un seul abonnement Limité ou Illimité en cours à la fois — il paie déjà pour un titre valide. Les tickets simples restent cumulables, comme un carnet de tickets. - Consommation d'un voyage : refusée si le titre est expiré, épuisé, suspendu ou résilié, avec le motif exact renvoyé au guichet. - Un scan rejoué (même identifiant de validation, par exemple après un retry réseau) ne décompte jamais deux fois le même voyage. - Vérification de la validité d'un titre (`GET /valide/:utilisateurId`) : c'est l'endpoint que consommera le futur Service Billetterie à chaque contrôle de QR Code. - Ticket simple : QR Code unique généré sur la fiche détail tant que le billet est actif, conformément au cahier des charges ; disparaît une fois consommé.

Cohérence commerciale — si ça casse, le catalogue ou la facturation deviennent incohérents : - Une formule n'est jamais supprimée, seulement désactivée (retirée du catalogue) — pour ne pas casser l'historique des abonnements déjà souscrits dessus. - Dès qu'une formule a au moins un abonnement, son tarif, sa durée de validité et son nombre de voyages sont figés ; seuls le nom et la description restent modifiables. - Une résiliation est définitive : impossible de réactiver ou de renouveler un abonnement résilié — sinon la règle du "un seul abonnement en cours" serait contournable.

Confort d'utilisation — utile, mais sans risque si ça ne marche pas : - Recherche et filtres : par statut, par type de titre, par client, et un filtre "expire sous N jours" pour anticiper les renouvellements. - Tableau de bord : total, répartition par statut et par type, voyages consommés, revenu total, abonnements qui expirent sous 7 jours.

Le choix de ces fonctionnalités de recherche et de tableau de bord répond à la consigne du professeur de justifier ce qui entre dans ces deux modules — le détail est dans [PLAN-SERVICE-ABONNEMENTS.md §8](#).

2. Plan de tests

Même principe qu'au premier TP, un outillage par côté :

- **Back :** `node:test` + `supertest` , sur une base MySQL dédiée aux tests. Les jetons sont signés avec le même secret que le Service Utilisateurs sans jamais s'y connecter — ça vérifie que l'interopérabilité

entre les deux services fonctionne réellement, pas seulement sur le papier.

- **Front** : Jest, sur `utils/validatorsAbonnements.js` (règles de formulaire). Le client API (`services/apiAbonnements.js`) fait de vrais appels HTTP vers le backend réel — comme `services/api.js` pour le Service Utilisateurs, il n'est pas testé unitairement ; sa logique est couverte par les tests du backend qu'il appelle.

```
# backend (service-abonnements/)
npm test                # toute la suite : 75 tests
npm run test:unitaires  # 24 tests, sans base de données
npm run test:api        # 51 tests, avec une base MySQL de test

# frontend
npm test                # 34 tests, dont 12 sur le périmètre Abonnements
```

Rendu des pages et interactions vérifiés manuellement, comme au premier TP.

3. Tableau de synthèse

[Back](#) — 75 tests, tous verts

ID	Cas de test	Type	Résultat attendu
Formules — création	Contrat respecté, voyages forcés selon le type, saisie invalide	API	201 conforme, 400 sur saisie invalide
Formules — consultation	Tableau simple, filtres type/actif, 404 sur formule inconnue	API	Résultats filtrés, 404 correct
Formules — modification	Libre tant que personne n'a souscrit, figée dès la première souscription, renommage toujours possible	API	200 ou 409 selon le champ et l'historique
Formules — activation	Retire/remet au catalogue, refuse une valeur non booléenne	API	200 ou 400
Souscriptions — création	Calcule expiration/solde, refuse saisie invalide, formule inconnue ou retirée	API	201, 400 ou 404/409 selon le cas
Souscriptions — un seul abonnement en cours	Refuse un second Limité/Illimité, autorise plusieurs tickets simples, n'affecte pas un autre client, ré-autorisé une fois expiré	API	409 ou 201 selon la règle
Souscriptions — consultation et filtres	Statuts recalculés avant filtrage, filtre client/type/expiration proche, filtre invalide refusé	API	Résultats corrects, 400 sur filtre invalide
Souscriptions — cycle de vie	Suspendre/réactiver, interdiction d'imposer un statut calculé, résiliation définitive, renouvellement, ticket simple non renouvelable	API	200, 400 ou 409 selon le cas
Consommation	Décompte, passage à EPUISE, scan rejoué sans double décompte, atomicité sur des scans simultanés, motif exact du refus, illimité jamais bloqué	API	Solde correct, 409 avec message explicite
Droit à voyager	Trois champs exacts du contrat, refus sans titre valide, titre qui expire le plus tôt retenu, ouvert aux agents pas aux clients	API	Réponse conforme au contrat
Tableau de bord	Toutes les clés même à vide, ventilation par statut/type, cumul des montants, réservé aux administrateurs	API	Valeurs correctes, 403 hors admin
Authentification	Jeton du Service Utilisateurs accepté, jeton absent/malformé/expiré	API	200/401 selon le cas

ID	Cas de test	Type	Résultat attendu
	refusé, route de vie sans authentification		
Solde de voyages	Déduction, jamais négatif, <code>null</code> pour un illimité	Unitaire	Valeurs correctes
Statut effectif	ACTIF/EXPIRE/EPUISE calculés correctement, priorité suspension > expiration > épuisement	Unitaire	Statut correct selon les priorités
Validation des modèles	Cohérence type/nombre de voyages, tarif et durée valides, identifiant client au format MongoDB	Unitaire	Erreurs de validation levées correctement

51 tests API + 24 tests unitaires.

Front — 34 tests (dont 12 sur le périmètre Abonnements)

Fichier	Cas couverts	Nb
<code>utils/validatorsAbonnements.test.js</code>	Formulaire de formule (nom, type, tarif, durée, voyages) et de souscription (client, formule, date)	12

Le reste (22 tests) porte sur le périmètre Service Utilisateurs, inchangé depuis le premier TP.

4. Scénario fonctionnel complet

De la création d'une formule à la consommation des voyages, écran par écran.

1. Un administrateur se connecte, ouvre **Formules** et crée un abonnement mensuel de 20 voyages à 15 000 FCFA.
2. Il ouvre **Abonnements** → **Nouvelle souscription**, choisit un client actif dans la liste et cette formule. L'abonnement est créé avec une date d'expiration à 30 jours et un solde de 20 voyages.
3. Il tente de souscrire le même client à une autre formule Limitée : refusé — le client a déjà un abonnement en cours. Il souscrit en revanche le même client à un Ticket simple : accepté, les tickets restent cumulables.
4. Sur la fiche détail de l'abonnement mensuel, il enregistre plusieurs voyages : le solde diminue à chaque fois, l'historique se remplit. Rejouer le même identifiant de validation ne décompte pas deux fois.
5. Il suspend l'abonnement : la consommation d'un nouveau voyage est refusée avec le motif "Abonnement suspendu". Il le réactive : la consommation redevient possible.
6. Il résilie l'abonnement : action définitive, ni réactivation ni renouvellement ne sont plus possibles ensuite. Le client peut désormais souscrire à un nouvel abonnement Limité ou Illimité.
7. Il consulte le **Tableau de bord** : total, répartition par statut et par type, voyages consommés, revenu total et abonnements expirant sous 7 jours reflètent exactement les actions effectuées.

5. Anomalies trouvées

Contrairement au premier TP, le code de ce service a été écrit avec les tests en parallèle plutôt qu'ajoutés après coup — moins de bugs applicatifs sont ressortis de ce côté. En revanche, la rédaction de cette documentation a servi de prétexte à comparer ligne à ligne le simulateur front (`apiAbonnements.js`) avec la vraie suite de tests du backend, alors que le premier annonçait respecter le contrat “au champ près”. Ça a fait remonter sept écarts réels, tous corrigés dans la même PR :

5.1 — Formule retirée du catalogue : mauvais code d'erreur. Le simulateur renvoyait 404 (“introuvable”) aussi bien pour une formule inexistante que pour une formule désactivée. Le backend distingue les deux : 404 pour “n'existe pas”, 409 pour “existe mais n'est plus proposée”. Corrigé.

5.2 — Aucune limite sur le nombre d'abonnements en cours. Le simulateur laissait un client souscrire à autant de formules Limitées ou Illimitées qu'il voulait en parallèle. Cette règle figurait dans le plan comme point ouvert (“un abonnement actif empêche-t-il d'en souscrire un second ?”), tranchée côté backend mais jamais reportée côté front. Corrigé : un seul Limité/Illimité en cours par client, les tickets simples restent cumulables.

5.3 — Tarif d'une formule modifiable après souscription. Le plan proposait initialement de ne pas autoriser la modification et de créer une nouvelle formule à la place ; le backend a finalement choisi de figer tarif/durée/nombre de voyages dès la première souscription tout en laissant le nom et la description modifiables. Le simulateur, lui, laissait tout modifier sans restriction. Corrigé pour suivre ce que fait réellement le backend.

5.4 — Une résiliation n'était pas définitive. Le simulateur permettait de repasser un abonnement résilié à `ACTIF`, ou de le renouveler — ce qui aurait permis de contourner la règle du point 5.2 (résilier puis réactiver pour cumuler des abonnements). Corrigé : `RESILIE` est un état terminal.

5.5 — Un ticket simple pouvait être “renouvelé”. Un ticket simple est un titre à usage unique ; le concept de renouvellement ne s'applique qu'aux formules à durée. Le simulateur ne faisait pas la différence. Corrigé.

5.6 — Un scan rejoué décomptait deux fois. Le simulateur ne gardait aucune trace des identifiants de validation déjà traités : renvoyer deux fois la même requête (retry réseau, double lecture du QR Code) décomptait deux voyages au lieu d'un. C'est exactement le genre de bug qui ne se voit pas en testant à la main une seule fois. Corrigé par une vérification d'unicité avant décompte.

5.7 — Le filtre “expire sous N jours” n'existait pas côté front. La liste des abonnements ne pouvait pas être filtrée sur les expirations proches, alors que c'est une des quatre fonctionnalités de recherche justifiées au §8.1 du plan. Ajouté.

Le point commun avec les anomalies du premier TP : dans les deux cas, c'est la confrontation entre deux sources indépendantes (le simulateur écrit d'un côté, les tests réels du backend de l'autre) qui a fait remonter les écarts — pas la relecture du code seul.

5.8 — Le front a ensuite été branché sur le vrai backend (plus de simulateur), et un dernier écart est ressorti à cette occasion : `verifierValidite` renvoyait l'abonnement complet (10 champs) au lieu des 3 prévus au contrat §4.4. Repéré par le développeur back en comparant le simulateur au sien avant même le branchement, corrigé avant que ça n'ait d'impact (aucun écran ne consommait encore cet endpoint). Le cycle complet a ensuite été rejoué en conditions réelles (souscription, consommation, historique, validité, statistiques) sans qu'aucun autre écart ne ressorte.

6. Limites connues

- Pas de test de bout en bout automatisé reliant le front au vrai backend — le cycle complet a été rejoué manuellement (voir §5.8), mais rien ne le rejoue automatiquement à chaque changement.

- Service Billetterie / QR Code (scan et validation) non développé à ce stade — c'est lui qui consommera `GET /validite/:utilisateurId` et `POST /souscriptions/:id/consommer`. Seule la génération du QR Code sur un ticket simple est faite côté front (§1), le contenu encodé reste volontairement simple (identifiant + type), sans jeton signé.
- `service-abonnements` doit tourner en permanence (MySQL compris) pour que les pages Formules, Abonnements et Tableau de bord fonctionnent — ce n'était pas le cas avant le branchement, quand le front tournait contre le simulateur.

7. Traçabilité

Élément	Emplacement
Contrat d'API	<code>PLAN-SERVICE-ABONNEMENTS.md</code>
Application Express (back)	<code>service-abonnements/src/app.js</code>
Modèles Sequelize	<code>service-abonnements/src/models/</code>
Tests API back	<code>service-abonnements/tests/api/</code>
Tests unitaires back	<code>service-abonnements/tests/unitaires/</code>
Client API (front)	<code>frontend/src/services/apiAbonnements.js</code>
Règles de validation front	<code>frontend/src/utils/validatorsAbonnements.js</code>
Pages Formules	<code>frontend/src/pages/FormulesManagement.jsx</code> , <code>frontend/src/components/FormuleModal.jsx</code>
Pages Abonnements	<code>frontend/src/pages/AbonnementsManagement.jsx</code> , <code>frontend/src/pages/AbonnementDetail.jsx</code> , <code>frontend/src/components/SouscriptionModal.jsx</code>
Tableau de bord	<code>frontend/src/pages/AbonnementStats.jsx</code>
Catalogue de démo	<code>service-abonnements/src/seed/seedFormules.js</code>